

Student Performance — Model Training

Complete Code Guide with Line-by-Line Explanations

This document walks through every code group in the Model Training phase of the Student Performance ML project. Each section shows the code block followed by a detailed line-by-line explanation — what it does, why it is used, and how it fits into the end-to-end machine learning pipeline.

Step 1**Importing Libraries — Basic & ML**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import mean_squared_error, r2_score
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor
from sklearn.svm import SVR
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.model_selection import RandomizedSearchCV
from catboost import CatBoostRegressor
from xgboost import XGBRegressor
import warnings
```

import numpy as np

NumPy provides fast array math. Aliased as np. Used here for np.sqrt() when computing RMSE from MSE.

import pandas as pd

Pandas is the core data library. Aliased as pd. Used to load the CSV, create feature/target variables, and build the results comparison DataFrame.

**import matplotlib.pyplot as plt /
seaborn as sns**

Visualization libraries for plotting actual vs predicted scatter plots and regression lines after training.

**from sklearn.metrics import
mean_squared_error, r2_score**

Imports evaluation metric functions. mean_squared_error computes MSE; r2_score measures how well the model explains variance in the target (1.0 = perfect).

**from sklearn.neighbors import
KNeighborsRegressor**

K-Nearest Neighbors Regressor — predicts a value by averaging the K closest training examples. Simple but can struggle with high-dimensional data.

**from sklearn.tree import
DecisionTreeRegressor**

Decision Tree — splits data by feature thresholds to form a tree of if-else rules. Prone to overfitting on training data if not constrained.

**from sklearn.ensemble import
RandomForestRegressor,
AdaBoostRegressor**

Ensemble methods that combine multiple models. RandomForest averages many decision trees (bagging). AdaBoost sequentially corrects errors of previous weak learners (boosting).

from sklearn.svm import SVR

Support Vector Regressor — finds the best hyperplane that fits the data within a margin. Imported but not used in the final model loop shown in the notebook.

**from sklearn.linear_model import
LinearRegression, Ridge, Lasso**

Linear models. LinearRegression is plain OLS. Ridge adds L2 regularization (penalizes large coefficients). Lasso adds L1 regularization (can zero out irrelevant features entirely).

from sklearn.model_selection import RandomizedSearchCV	Hyperparameter tuning tool. Randomly samples combinations from a parameter grid and cross-validates each — faster than exhaustive GridSearchCV. Imported for potential tuning use.
from catboost import CatBoostRegressor	CatBoost is a gradient boosting library by Yandex, optimized for categorical features. verbose=False suppresses its verbose training output.
from xgboost import XGBRegressor	XGBoost (Extreme Gradient Boosting) is one of the most powerful and widely used boosting algorithms in ML competitions and production models.

Insight

Nine different regression algorithms are imported, ranging from simple linear models to complex ensemble/boosting methods. Training all of them in one loop lets us pick the best performer objectively.

Step 2

Loading Data & Defining Target Variable

```
df = pd.read_csv('data/stud.csv')
df.head()
```

```
# Target variable
y = df['math_score']
```

df = pd.read_csv('data/stud.csv')	Reads the student CSV file from disk into a Pandas DataFrame. All 1000 rows and 8 columns are loaded into memory as df.
df.head()	Displays the first 5 rows as a quick sanity check — confirms columns loaded correctly and data looks as expected before any preprocessing.
y = df['math_score']	Extracts the target variable (the value we want to predict) as a Pandas Series. y contains the math score for all 1000 students. This is what all models will be trained to predict.

Insight

In supervised ML, we always separate X (features/inputs) and y (target/output) before training. Here y = math_score, meaning the model learns to predict maths performance from all other student attributes.

Step 3**Preparing Feature Matrix X & Checking Categories**

```
X = df.drop(columns=['math_score'], axis=1)

print(df['gender'].unique())
print(df['race_ethnicity'].unique())
print(df['parental_level_of_education'].unique())
print(df['lunch'].unique())
print(df['test_preparation_course'].unique())
```

**X =
df.drop(columns=['math_score'],
axis=1)**

Creates the feature matrix X by removing the target column 'math_score' from the DataFrame. axis=1 specifies we're dropping a column (not a row). X now has 7 columns: all student attributes except the score we're predicting.

df['gender'].unique()

Prints all distinct values in each categorical column. Done here to verify the exact category strings before encoding — any typos or unexpected values would cause issues in preprocessing.

print(...) for each column

Confirms: gender has 2 categories, race_ethnicity has 5, parental education has 6, lunch has 2, test_preparation_course has 2. Total categorical levels inform the expected feature count after encoding.

Insight

X contains the 7 input features used to predict math_score. The category inspection is essential before OneHotEncoding — it tells us exactly how many binary columns will be created (one per category per feature).

Step 4

Preprocessing: OneHotEncoding + StandardScaler

```
num_features = X.select_dtypes(exclude='object').columns
cat_features = X.select_dtypes(include='object').columns

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

numeric_transformer = StandardScaler()
oh_transformer = OneHotEncoder()

preprocessor = ColumnTransformer([
    ('OneHotEncoder', oh_transformer, cat_features),
    ('StandardScaler', numeric_transformer, num_features),
])

X = preprocessor.fit_transform(X)
X.shape # Output: (1000, 19)
```

X.select_dtypes(exclude='object').columns	Selects only numeric columns (those not of type 'object'). Result: reading_score and writing_score — the two non-categorical input features.
X.select_dtypes(include='object').columns	Selects only categorical text columns. Result: gender, race_ethnicity, parental_level_of_education, lunch, test_preparation_course — the 5 string columns.
StandardScaler()	Creates a scaler that transforms numeric values to have mean=0 and std=1. This prevents features with larger ranges from dominating distance-based models like KNN.
OneHotEncoder()	Creates an encoder that converts each category into a binary column (0 or 1). For example, gender becomes two columns: gender_female and gender_male. Needed because ML models require numeric input.
ColumnTransformer([...])	Applies different transformers to different column subsets in one step. OneHotEncoder goes to cat_features; StandardScaler goes to num_features. Remaining columns are handled as specified.
preprocessor.fit_transform(X)	fit() learns the encoding/scaling parameters from X (e.g., mean, std, unique categories). transform() applies those parameters to convert X into a numeric matrix. Combined as fit_transform() for efficiency.
X.shape -> (1000, 19)	Confirms the transformation produced 1000 rows and 19 columns. The 5 categorical columns expanded into 17 binary columns via OHE, plus 2 scaled numeric columns = 19 total features.

Insight

Preprocessing is mandatory before model training. Raw text categories crash most sklearn models. StandardScaling ensures numeric features contribute equally — without it, writing_score (0-100) would outweigh a binary OHE column.

Step 5

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape # Output: ((800, 19), (200, 19))
```

**from sklearn.model_selection
import train_test_split**

Imports the function that randomly splits data into training and testing subsets. Essential for evaluating how well models generalize to unseen data.

**X_train, X_test, y_train, y_test =
train_test_split(...)**

Splits both X (features) and y (target) simultaneously. Returns 4 arrays: X_train and y_train for fitting the model; X_test and y_test for evaluating it on unseen data.

test_size=0.2

Reserves 20% of data (200 rows) for testing and uses the remaining 80% (800 rows) for training. A common and well-accepted split ratio in ML practice.

random_state=42

Seeds the random number generator to ensure the split is identical every time the code runs. Without this, you'd get a different split each run, making results non-reproducible.

**X_train.shape, X_test.shape ->
((800, 19), (200, 19))**

Confirms the split: 800 training samples and 200 test samples, each with 19 features. y_train and y_test have the corresponding math scores.

Insight

Without a train-test split, you have no way to tell if a model is overfitting (memorizing training data) vs genuinely learning patterns. The test set acts as a final unseen exam for each model.

Step 6**Evaluation Function**

```
def evaluate_model(true, predicted):  
    mae = mean_absolute_error(true, predicted)  
    mse = mean_squared_error(true, predicted)  
    rmse = np.sqrt(mean_squared_error(true, predicted))  
    r2_square = r2_score(true, predicted)  
    return mae, rmse, r2_square
```

def evaluate_model(true, predicted):

Defines a reusable function that accepts actual values (true) and model predictions (predicted), then returns three evaluation metrics. Avoids repeating the same metric calculations for every model.

mae = mean_absolute_error(true, predicted)

Mean Absolute Error — the average of absolute differences between actual and predicted values. Intuitive: on average, predictions are off by this many marks. Less sensitive to large errors than RMSE.

mse = mean_squared_error(true, predicted)

Mean Squared Error — squares each error before averaging. Larger errors are penalized more heavily. Used here to compute RMSE but not returned directly.

rmse = np.sqrt(mean_squared_error(true, predicted))

Root Mean Squared Error — the square root of MSE. Brings the error back to the same units as the target (marks). More interpretable than MSE and punishes large outlier predictions.

r2_square = r2_score(true, predicted)

R-squared (coefficient of determination) — measures what proportion of variance in math_score is explained by the model. Range: 0 to 1. Score of 0.88 means the model explains 88% of variance.

return mae, rmse, r2_square

Returns all three metrics as a tuple. The caller can unpack them into named variables for printing and comparison across models.

Insight

A dedicated evaluation function promotes clean, DRY (Don't Repeat Yourself) code. All 9 models use the same function, ensuring metrics are computed identically and are directly comparable.

Step 7**Defining All Models in a Dictionary**

```
models = {  
    "Linear Regression": LinearRegression(),  
    "Lasso": Lasso(),  
    "Ridge": Ridge(),  
    "K-Neighbors Regressor": KNeighborsRegressor(),  
    "Decision Tree": DecisionTreeRegressor(),  
    "Random Forest Regressor": RandomForestRegressor(),  
    "XGBRegressor": XGBRegressor(),  
    "CatBoosting Regressor": CatBoostRegressor(verbose=False),  
    "AdaBoost Regressor": AdaBoostRegressor()  
}  
model_list = []  
r2_list = []
```

models = { "Name": ModelClass() }

Stores all 9 model objects in a Python dictionary. Keys are human-readable model names (strings); values are instantiated sklearn/catboost/xgboost model objects with default hyperparameters.

LinearRegression()

Plain Ordinary Least Squares linear regression. No regularization. Best baseline model — if complex models don't beat this, they may be overfitting.

Lasso() / Ridge()

Regularized linear models. Lasso (L1) can shrink some feature coefficients to exactly zero — useful for feature selection. Ridge (L2) shrinks all coefficients smoothly without eliminating them.

KNeighborsRegressor()

Predicts by averaging the y values of the K nearest training points. No training phase — all computation happens at prediction time. Sensitive to feature scaling (why StandardScaler was applied).

DecisionTreeRegressor()

Builds a binary tree of if-else rules. Very flexible — can perfectly fit training data, making it prone to overfitting. Typically has a large gap between train and test R2.

RandomForestRegressor()

Trains many decision trees on random data subsets and averages their predictions. Reduces overfitting vs a single tree. Generally robust and high-performing out of the box.

**XGBRegressor() /
CatBoostRegressor(verbose=False)**

Gradient boosting implementations — sequentially build trees where each one corrects the errors of the previous. Typically among the best performers. verbose=False silences CatBoost's training log output.

AdaBoostRegressor()

Adaptive Boosting — assigns higher weights to misclassified samples so the next model focuses on them. Simpler boosting method than XGBoost but still competitive on tabular data.


```
model_list = [] / r2_list = []
```

Empty lists initialized to collect model names and their R2 test scores during the training loop. Used at the end to build a comparison DataFrame.

Insight

Using a dictionary of models allows all 9 algorithms to be trained, evaluated, and compared in a single loop. This pattern is a standard best practice in ML experimentation.

Step 8

Training Loop — Fit, Predict & Evaluate All Models

```
for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    model_train_mae, model_train_rmse, model_train_r2 = evaluate_model(y_train, y_train_pred)
    model_test_mae, model_test_rmse, model_test_r2 = evaluate_model(y_test, y_test_pred)

    print(list(models.keys())[i])
    model_list.append(list(models.keys())[i])

    print('Model performance for Training set')
    print('- Root Mean Squared Error: {:.4f}'.format(model_train_rmse))
    print('- Mean Absolute Error: {:.4f}'.format(model_train_mae))
    print('- R2 Score: {:.4f}'.format(model_train_r2))
    print('Model performance for Test set')
    print('- Root Mean Squared Error: {:.4f}'.format(model_test_rmse))
    print('- Mean Absolute Error: {:.4f}'.format(model_test_mae))
    print('- R2 Score: {:.4f}'.format(model_test_r2))
    r2_list.append(model_test_r2)
```

for i in range(len(list(models))):	Loops over indices 0 through 8, one for each model in the dictionary. len(list(models)) = 9, so i goes from 0 to 8.
model = list(models.values())[i]	Retrieves the model object at position i from the dictionary values. For i=0, this is LinearRegression(); for i=1, Lasso(); and so on.
model.fit(X_train, y_train)	Trains the model on the training data. fit() adjusts the model's internal parameters to best predict y_train from X_train. This is where 'learning' actually happens.
y_train_pred = model.predict(X_train)	Generates predictions on the training set after fitting. Used to compute training metrics — helps detect overfitting (if train metrics are much better than test metrics).
y_test_pred = model.predict(X_test)	Generates predictions on the held-out test set. The model has never seen these samples during training. These predictions measure real-world generalization ability.
evaluate_model(y_train, y_train_pred)	Calls the custom function to compute MAE, RMSE, and R2 for the training set. Unpacked into three named variables for printing.
evaluate_model(y_test, y_test_pred)	Same evaluation function applied to test predictions. The test R2 is the key metric for comparing models — it reflects performance on truly unseen data.

```
print('- R2 Score:
{:.4f}'.format(...))
```

`{:.4f}` formats the float to exactly 4 decimal places for clean, aligned output. Makes it easy to visually compare all 9 models' results at a glance.

```
r2_list.append(model_test_r2)
```

Appends the test R2 score to the collection list after each model. At the end of the loop, `r2_list` holds all 9 test R2 scores, ready to be paired with model names for ranking.

Insight

Computing both train and test metrics reveals overfitting. Decision Tree had train R2=0.9997 but test R2=0.76 — a clear sign it memorized training data. Linear Regression had nearly identical train (0.874) and test (0.880) R2 — a well-generalizing model.

Step 9

Results: Model Comparison DataFrame

```
pd.DataFrame(
    list(zip(model_list, r2_list)),
    columns=['Model Name', 'R2_Score']
)
```

```
list(zip(model_list, r2_list))
```

`zip()` pairs each model name with its corresponding R2 score — `[("Linear Regression", 0.880), ("Lasso", 0.825), ...]`. `list()` converts the zip object into a list of tuples.

```
pd.DataFrame(..., columns=[...])
```

Creates a clean comparison table from the list of tuples. `columns=['Model Name', 'R2_Score']` labels the two columns. Sorted by `R2_Score`, this reveals the ranking of all 9 models at a glance.

Insight

Final rankings by test R2 score: Ridge (0.8806) > Linear Regression (0.8803) > CatBoosting (0.8516) > AdaBoost (0.8498) > Random Forest (0.8473) > Lasso (0.8253) > XGBoost (0.8216) > KNN (0.7838) > Decision Tree (0.7603). Simple linear models topped the leaderboard — a common outcome when the data has strong linear relationships.

Final Model: Linear Regression Deep Dive

```
lin_model = LinearRegression(fit_intercept=True)
lin_model = lin_model.fit(X_train, y_train)
y_pred = lin_model.predict(X_test)
score = r2_score(y_test, y_pred) * 100
print(' Accuracy of the model is %.2f' % score)

# Scatter plot: Actual vs Predicted
plt.scatter(y_test, y_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')

# Regression plot with fit line
sns.regplot(x=y_test, y=y_pred, ci=None, color='red')
```

LinearRegression(fit_intercept=True)	Instantiates the Linear Regression model. <code>fit_intercept=True</code> (the default) tells the model to also learn a bias/intercept term b in the equation: $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b$.
lin_model.fit(X_train, y_train)	Trains the model on the 800-sample training set. Internally, it computes the optimal weight vector using the Ordinary Least Squares formula that minimizes the sum of squared residuals.
y_pred = lin_model.predict(X_test)	Applies the learned weights to each of the 200 test feature vectors and returns predicted math scores. These are the model's guesses for students it has never seen.
score = r2_score(y_test, y_pred) * 100	Computes R^2 and multiplies by 100 to express it as a percentage. Score of 88.03 means the model explains 88.03% of the variance in test math scores — a strong result.
print('Accuracy of the model is %.2f' % score)	<code>%.2f</code> is a classic Python format specifier meaning: print the float with exactly 2 decimal places. Outputs: 'Accuracy of the model is 88.03'.
plt.scatter(y_test, y_pred)	Plots actual scores on x-axis vs predicted scores on y-axis. If the model were perfect, all points would fall on a diagonal line from bottom-left to top-right. Points close to the diagonal = accurate predictions.
sns.regplot(x=y_test, y=y_pred, ci=None, color='red')	Seaborn's regression plot draws the scatter points AND fits a regression line through them. <code>ci=None</code> removes the confidence interval shading for a cleaner chart. The near-diagonal line confirms strong linear prediction.

Insight

Linear Regression with `fit_intercept` achieves 88.03% R^2 on the test set. The scatter and regression plots visually confirm tight alignment between actual and predicted scores — with only a few outliers at the extremes.

**Step
11****Actual vs Predicted Difference Table**

```
pred_df = pd.DataFrame({  
    'Actual Value': y_test,  
    'Predicted Value': y_pred,  
    'Difference': y_test - y_pred  
})  
pred_df
```

pred_df = pd.DataFrame({...})

Creates a 3-column DataFrame for the 200 test samples. Passing a dictionary maps column names to data arrays. The index comes from y_test's original row indices (student IDs from the original df).

'Actual Value': y_test

The real math scores from the test set — ground truth values the model is trying to predict.

'Predicted Value': y_pred

The math scores predicted by the Linear Regression model. These are continuous floats because linear regression outputs a real-valued number, not an integer.

'Difference': y_test - y_pred

The residual — actual minus predicted. Positive difference means the model under-predicted; negative means it over-predicted. Analyzing residuals reveals systematic errors or patterns.

pred_df

Displays the full 200-row table. Useful for spotting individual students where the model was very wrong (e.g., student 521: actual=91, predicted=76.5, difference=14.5). Helps identify edge cases.

Insight

Most differences are small (1-6 marks), showing the model is well-calibrated. The largest errors tend to occur at the extremes — very high or very low scorers — a common pattern when training data has fewer extreme examples.

Quick Reference Summary

Step	Operation	Key Functions / Classes	Purpose
1	Import Libraries	sklearn, catboost, xgboost	Load all ML and viz tools
2	Load Data & Target	pd.read_csv(), df['math_score']	Read CSV; extract y (target)
3	Feature Matrix X	df.drop(), df['col'].unique()	Remove target; verify categories
4	Preprocessing	OneHotEncoder, StandardScaler, ColumnTransformer, fit_transform()	Encode categories; scale numerics
5	Train-Test Split	train_test_split(test_size=0.2)	Reserve 20% for unbiased evaluation

6	Evaluation Function	mean_absolute_error, mean_squared_error, r2_score, np.sqrt()	Reusable MAE / RMSE / R2 calculator
7	Model Dictionary	LinearRegression, Ridge, Lasso, KNN, DecisionTree, RandomForest, XGB, CatBoost, AdaBoost	All 9 models in one structure
8	Training Loop	model.fit(), model.predict(), evaluate_model()	Train, predict, and score all models
9	Results DataFrame	zip(), pd.DataFrame()	Rank all models by test R2
10	Final Model Deep Dive	LinearRegression, plt.scatter(), sns.regplot()	Retrain best model; visualize fit
11	Prediction Difference	pd.DataFrame({'Difference': y_test-y_pred})	Inspect residuals per student